

Data Structure ver_2.0

Just Pick It Data Structure

1. 데이터 구조 개요

이 문서는 Just Pick It 시스템에서 사용하는 주요 DB 테이블, ENUM, 테이블 간 관계를 정의한다.

Fleet Manager는 주문, 재고, 로봇 상태, 작업 상태, 예외 기록을 저장하고 UI에 제공한다.

Fleet Manager는 저장된 주문/상품/로봇 정보를 기반으로 task를 생성하고, task 실행 결과를 Fleet API/WebSocket을 통해 Web Gateway에 제공한다.

발표/시나리오 관점에서는 PICKY를 “AMR + COBOT이 함께 동작하는 작업 세트”로 설명한다.

실제 구현/DB 관점에서는 PICKY와 COBOT을 개별 robot으로 저장하고, `robot_unit` 으로 두 로봇의 작업 세트를 연결한다.

주문에 포함된 상품 목록은 `order_item` 에 저장한다.

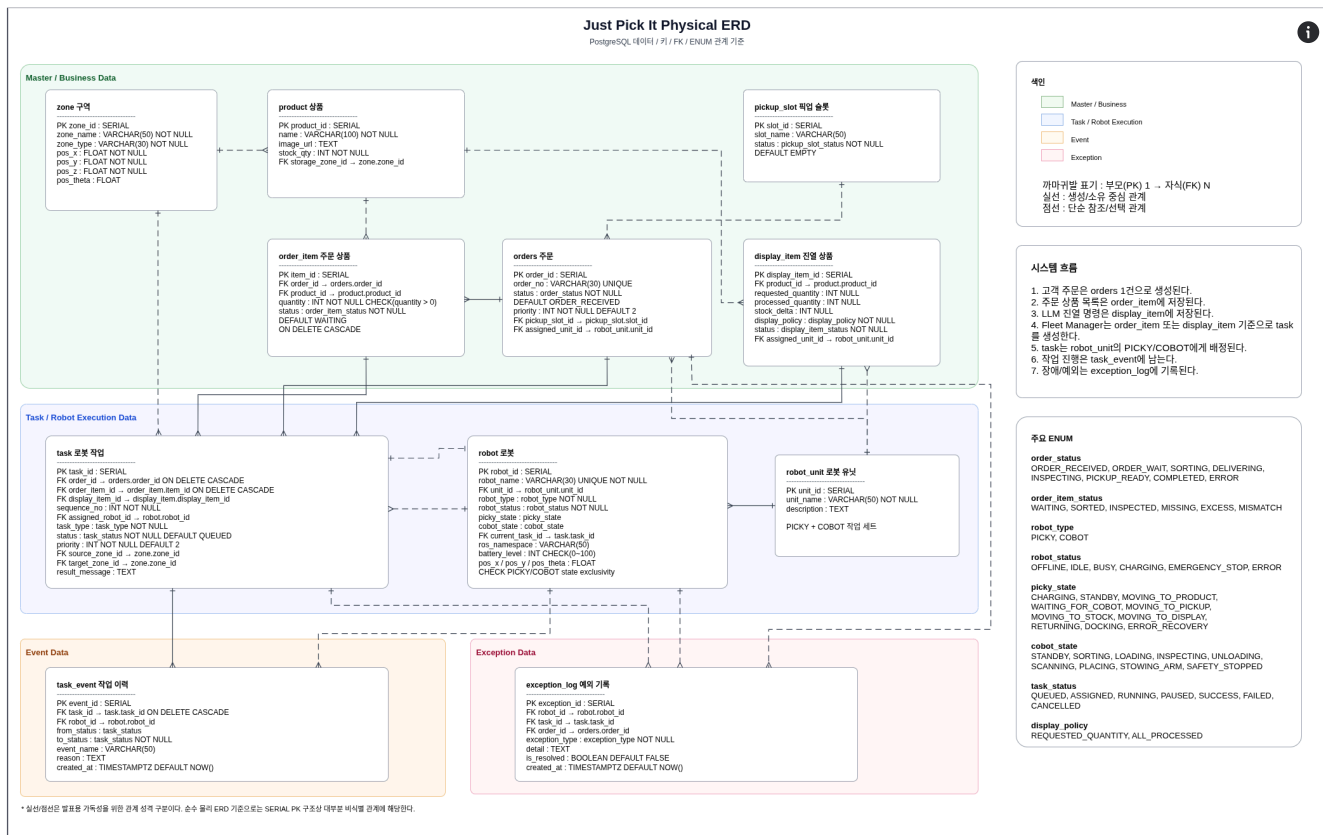
Fleet Manager가 생성한 task는 `order_item_id` 를 통해 특정 주문 상품과 연결된다.

따라서 `order_item` 은 “주문 원본 상품 목록”이고, `task` 는 “로봇이 실행할 작업 순서”이다.

진열은 주문과 다르게 `order_item` 을 만들지 않는다.

진열 대상 상품, 요청 수량, 실제 처리 완료 수량, 재고 증가 수량은 `display_item` 에 저장한다.

2. ERD 개요



핵심은 고객 주문과 진열 요청을 Fleet Manager가 실행 가능한 robot task로 분해하고, 각 task를 PICKY와 COBOT의 실제 상태 변화와 연결해 관리하는 구조이다.

Fleet Manager는 주문, 재고, 로봇 상태, task 상태, 예외 기록을 저장한다.

Fleet Manager는 TaskManager와 TrafficManager를 포함하며, DB 읽기/쓰기, 실제 task 생성, task 배정, 경로 충돌 판단, State Machine 명령 전달을 담당한다.

PICKY와 COBOT은 Web Gateway와 직접 통신하지 않고 Fleet Manager를 통해 제어된다.

3. 테이블 역할 요약

테이블	역할	사용 위치
zone	로봇이 이동하거나 작업하는 위치 정보	Fleet Manager, TrafficManager, PICKY 목적지, UI 경로 표시
product	상품 정보와 재고	고객 상품 목록, 관리자 재고 관리
pickup_slot	고객 픽업 슬롯 상태	픽업존, 고객 수령

orders	주문 1건의 대표 상태	고객 주문 상태, 관리자 주문 현황
order_item	주문에 포함된 상품 목록	상품별 상차 task, 검수 기준
display_item	진열 대상 상품과 수량 정보	LLM 진열 명령, 진열 task, 재고 증가
robot_unit	함께 동작하는 PICKY와 COBOT 묶음	Fleet Manager task 배정, 같은 세트 로봇 조회
robot	PICKY / COBOT 개별 로봇 상태	관리자 관제, Fleet Manager 상태 관리
task	Fleet Manager가 로봇에게 줄 작업 단위	작업 큐, task 배정, task 진행 상태
task_event	작업 진행 이력	관리자 작업 타임라인, 디버깅
exception_log	예외 상황 기록 및 관리자 알림	장애 분석, 관리자 대시보드

4. ENUM 타입 정의

```

1 CREATE TYPE order_status AS ENUM (
2     'ORDER_RECEIVED', -- 주문 접수
3     'ORDER_WAIT',    -- 주문 대기
4     'SORTING',       -- 상품 선별/상차 중
5     'DELIVERING',    -- 상품 운반 중
6     'INSPECTING',    -- 상품 검수/하차 중
7     'PICKUP_READY',  -- 픽업 준비 완료
8     'COMPLETED',    -- 수령 완료
9     'ERROR'         -- 예외 발생
10 );
11
12 CREATE TYPE order_item_status AS ENUM (
13     'WAITING', -- 작업 대기 중
14     'SORTED', -- 상품 선별 및 PICKY 적재 완료
15     'INSPECTED', -- 상품 검수 완료
16     'MISSING', -- 검수 결과 상품 누락
17     'EXCESS', -- 검수 결과 상품 초과
18     'MISMATCH' -- 검수 결과 불일치
19 );
20
21 CREATE TYPE pickup_slot_status AS ENUM (
22     'EMPTY', -- 비어 있음
23     'RESERVED', -- 주문에 배정됨
24     'OCCUPIED', -- 상품 있음 / 고객 픽업 대기
25     'BLOCKED' -- 사용 불가
26 );
27

```

```

28 CREATE TYPE robot_type AS ENUM (
29     'PICKY',      -- 주행 로봇
30     'COBOT'       -- 로봇팔
31 );
32
33 CREATE TYPE robot_status AS ENUM (
34     'OFFLINE',    -- 통신 단절
35     'IDLE',        -- 대기 중 / 신규 작업 가능
36     'BUSY',        -- 작업 수행 중 / 신규 작업 불가
37     'CHARGING',    -- 충전 중
38     'EMERGENCY_STOP', -- 긴급 정지 상태
39     'ERROR'        -- 로봇 오류 상태
40 );
41
42 CREATE TYPE picky_state AS ENUM (
43     'CHARGING',
44     'STANDBY',
45     'MOVING_TO_PRODUCT',
46     'WAITING_FOR_COBOT',
47     'MOVING_TO_PICKUP',
48     'MOVING_TO_STOCK',
49     'MOVING_TO_DISPLAY',
50     'RETURNING',
51     'DOCKING',
52     'ERROR_RECOVERY'
53 );
54
55 CREATE TYPE cobot_state AS ENUM (
56     'STANDBY',
57     'SORTING',
58     'LOADING',
59     'INSPECTING',
60     'UNLOADING',
61     'SCANNING',
62     'PLACING',
63     'STOWING_ARM',
64     'SAFETY_STOPPED'
65 );
66
67 CREATE TYPE task_type AS ENUM (
68     'MOVE_TO_PRODUCT',
69     'SORTING_AND_LOAD',
70     'MOVE_TO_PICKUP',
71     'INSPECTION',
72     'UNLOAD',
73     'MOVE_TO_STOCK',
74     'MOVE_TO_DISPLAY',
75     'DISPLAY_SCAN',
76     'DISPLAY_PLACE',
77     'RETURN_HOME',
78     'DOCK_IN',
79     'CHARGE'
80 );
81
82 CREATE TYPE task_status AS ENUM (
83     'QUEUED',
84     'ASSIGNED',
85     'RUNNING',
86     'PAUSED',
87     'SUCCESS',
88     'FAILED',
89     'CANCELLED'
90 );
91
92 CREATE TYPE exception_type AS ENUM (
93     'OBSTACLE_DETECTED',
94     'LOW_BATTERY',

```

```

95     'NAVIGATION_FAILED',
96     'HARDWARE_ERROR',
97     'TIMEOUT',
98     'SORTING_FAIL',
99     'INSPECTION_FAIL',
100    'HUMAN_DETECTED',
101    'SYSTEM_ERROR'
102 );
103
104 CREATE TYPE display_policy AS ENUM (
105     'REQUESTED_QUANTITY', -- 명령에 지정된 수량만 진열
106     'ALL_PROCESSED'       -- COBOT이 실제 처리 완료한 대상 상품 전체 진
107     열
108 );
109
110 CREATE TYPE display_item_status AS ENUM (
111     'REQUESTED',
112     'ASSIGNED',
113     'IN_PROGRESS',
114     'COMPLETED',
115     'FAILED',
116     'CANCELLED'
117 );

```

5. 테이블 정의

5-1. zone

zone 은 PICKY가 이동하거나 정차하는 주요 위치를 저장한다.

상품 보관 위치, 창고 구역, 진열 구역, 픽업존, 충전 도크, 교통 제어 구역 등을 모두 zone으로 관리한다.

```

1 CREATE TABLE zone (
2     zone_id      SERIAL PRIMARY KEY,
3     zone_name    VARCHAR(50) NOT NULL,
4     zone_type    VARCHAR(30) NOT NULL,
5     pos_x        FLOAT NOT NULL,
6     pos_y        FLOAT NOT NULL,
7     pos_z        FLOAT NOT NULL,
8     pos_theta    FLOAT
9 );

```

5-2. product

product 는 상품 정보와 재고를 저장한다.

각 상품은 하나의 보관 zone에 연결된다.

```

1 CREATE TABLE product (
2     product_id   SERIAL PRIMARY KEY,
3     name         VARCHAR(100) NOT NULL,
4     image_url    TEXT,
5     stock_qty    INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
6     storage_zone_id INT NOT NULL REFERENCES zone(zone_id)
7 );

```

5-3. pickup_slot

`pickup_slot` 은 고객이 상품을 가져갈 픽업 칸을 관리한다.

주문과의 연결은 `orders.pickup_slot_id` 로 관리한다.

```
1 CREATE TABLE pickup_slot (  
2     slot_id SERIAL PRIMARY KEY,  
3     slot_name VARCHAR(50),  
4     status pickup_slot_status NOT NULL DEFAULT 'EMPTY'  
5 );
```

5-4. robot_unit

`robot_unit` 은 함께 동작하는 PICKY와 COBOT 묶음이다.

한 주문 또는 진열 작업은 하나의 `robot_unit`에 배정된다.

Fleet Manager는 `unit_id` 를 기준으로 같은 세트의 PICKY와 COBOT을 찾는다.

```
1 CREATE TABLE robot_unit (  
2     unit_id SERIAL PRIMARY KEY,  
3     unit_name VARCHAR(50) NOT NULL,  
4     description TEXT  
5 );
```

5-5. orders

`orders` 는 주문 1건의 대표 정보를 저장한다.

`priority` 는 낮을수록 우선순위가 높다.

일반 주문의 기본 `priority`는 2이다.

`assigned_unit_id` 는 해당 주문을 담당하는 `robot_unit`을 의미한다.

```
1 CREATE TABLE orders (  
2     order_id SERIAL PRIMARY KEY,  
3     order_no VARCHAR(30) UNIQUE,  
4     status order_status NOT NULL DEFAULT 'ORDER_RECEIVED',  
5     priority INT NOT NULL DEFAULT 2,  
6     pickup_slot_id INT REFERENCES pickup_slot(slot_id),  
7     assigned_unit_id INT REFERENCES robot_unit(unit_id)  
8 );
```

5-6. order_item

`order_item` 은 주문 안에 들어 있는 상품 목록을 저장한다.

예를 들어 고객이 콜라 2개, 과자 1개를 주문하면 `orders` 는 1개 생성되고, `order_item` 은 2개 생성된다.

Fleet Manager는 `order_item` 을 기준으로 상품별 task를 만든다.

```
1 CREATE TABLE order_item (  
2     item_id SERIAL PRIMARY KEY,  
3     order_id INT NOT NULL REFERENCES orders(order_id) ON DELETE  
4     CASCADE,  
5     product_id INT NOT NULL REFERENCES product(product_id),  
6     quantity INT NOT NULL CHECK (quantity > 0),  
7     status order_item_status NOT NULL DEFAULT 'WAITING'  
8 );
```

5-7. display_item

display_item 은 LLM 진열 명령으로 생성되는 진열 대상 상품 정보를 저장한다.

진열은 고객 주문이 아니므로 orders 와 order_item 을 만들지 않는다.

수량이 명령에 포함되면 requested_quantity 를 사용한다.

수량이 명령에 없으면 requested_quantity 는 NULL이고, COBOT이 실제 처리 완료한 수량을 processed_quantity 에 저장한다.

```
1 CREATE TABLE display_item (  
2     display_item_id SERIAL PRIMARY KEY,  
3     product_id INT NOT NULL REFERENCES product(product_id),  
4     requested_quantity INT CHECK (  
5         requested_quantity IS NULL OR requested_quantity > 0  
6     ),  
7     processed_quantity INT CHECK (  
8         processed_quantity IS NULL OR processed_quantity >= 0  
9     ),  
10    stock_delta INT CHECK (  
11        stock_delta IS NULL OR stock_delta >= 0  
12    ),  
13    display_policy display_policy NOT NULL,  
14    status display_item_status NOT NULL DEFAULT  
15    'REQUESTED',  
16    assigned_unit_id INT REFERENCES robot_unit(unit_id),  
17    CHECK (  
18        (display_policy = 'REQUESTED_QUANTITY' AND requested_quantity  
19        IS NOT NULL)  
20        OR  
21        (display_policy = 'ALL_PROCESSED' AND requested_quantity IS  
22        NULL)  
23    )  
24 );
```

5-8. robot

robot 은 PICKY와 COBOT 개별 로봇의 현재 상태를 저장한다.

robot_status 는 관제용 큰 상태이고, picky_state 와 cobot_state 는 타입별 세부 상태이다.

PICKY는 picky_state 만 사용하고, COBOT은 cobot_state 만 사용한다.

```
1 CREATE TABLE robot (  
2     robot_id SERIAL PRIMARY KEY,  
3     robot_name VARCHAR(30) UNIQUE NOT NULL,  
4     unit_id INT REFERENCES robot_unit(unit_id),  
5     robot_type robot_type NOT NULL,  
6     robot_status robot_status NOT NULL DEFAULT 'IDLE',  
7     picky_state picky_state,  
8     cobot_state cobot_state,  
9     current_task_id INT,  
10    ros_namespace VARCHAR(50),  
11    battery_level INT CHECK (battery_level BETWEEN 0 AND 100),  
12    pos_x FLOAT,  
13    pos_y FLOAT,  
14    pos_theta FLOAT,  
15    CHECK (  
16        (robot_type = 'PICKY' AND cobot_state IS NULL)  
17        OR  
18        (robot_type = 'COBOT' AND picky_state IS NULL)  
19    )  
20 );
```

```
19 )
20 );
```

5-9. task

task 는 Fleet Manager가 생성하고 관리하는 작업 단위이다.

orders 가 고객 관점의 주문이라면, task 는 로봇 실행 관점의 명령이다.

상품별 이동/상차 작업은 order_item_id 를 기준으로 연결한다.

sequence_no 는 단순 생성 순번이 아니라 Fleet Manager가 상품 위치, PICKY 현재 위치, 경로 계산 결과를 기준으로 정한 실행 순서이다.

진열 task는 display_item_id 를 기준으로 연결한다.

진열 task에서는 order_id 와 order_item_id 가 NULL이다.

```
1 CREATE TABLE task (
2     task_id          SERIAL PRIMARY KEY,
3     order_id         INT REFERENCES orders(order_id) ON DELETE
4     CASCADE,
5     order_item_id    INT REFERENCES order_item(item_id) ON DELETE
6     CASCADE,
7     display_item_id  INT REFERENCES display_item(display_item_id) ON
8     DELETE CASCADE,
9     sequence_no      INT NOT NULL,
10    assigned_robot_id INT REFERENCES robot(robot_id),
11    task_type         task_type NOT NULL,
12    status            task_status NOT NULL DEFAULT 'QUEUED',
13    priority          INT NOT NULL DEFAULT 2,
14    source_zone_id    INT REFERENCES zone(zone_id),
15    target_zone_id    INT REFERENCES zone(zone_id),
16    result_message    TEXT,
17    CHECK (
18        NOT (order_item_id IS NOT NULL AND display_item_id IS NOT
19        NULL)
20    ),
21    CHECK (
22        display_item_id IS NULL
23        OR
24        (order_id IS NULL AND order_item_id IS NULL)
25    )
26 );
27
28 ALTER TABLE robot
29 ADD CONSTRAINT fk_robot_current_task
30 FOREIGN KEY (current_task_id) REFERENCES task(task_id);
```

5-10. task_event

task_event 는 작업 상태 변화와 주요 이벤트를 기록한다.

작업 할당, 작업 시작, 경로 허가, 이동 완료, COBOT 작업 완료, 실패 사유 등을 남긴다.

```
1 CREATE TABLE task_event (
2     event_id         SERIAL PRIMARY KEY,
3     task_id          INT NOT NULL REFERENCES task(task_id) ON DELETE
4     CASCADE,
5     robot_id         INT REFERENCES robot(robot_id),
6     from_status      task_status,
7     to_status        task_status NOT NULL,
8     event_name       VARCHAR(50),
```



```

8      reason      TEXT,
9      created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );

```

5-11. exception_log

exception_log 는 로봇이나 시스템에서 발생한 문제를 기록한다.

예외 발생 시점은 요구사항에 포함되어 있으므로 created_at 을 유지한다.

```

1 CREATE TABLE exception_log (
2     exception_id SERIAL PRIMARY KEY,
3     robot_id     INT REFERENCES robot(robot_id),
4     task_id      INT REFERENCES task(task_id),
5     order_id     INT REFERENCES orders(order_id),
6     exception_type exception_type NOT NULL,
7     detail       TEXT,
8     is_resolved  BOOLEAN NOT NULL DEFAULT FALSE,
9     created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
10 );

```

6. 데이터 관계 요약

관계	설명
product.storage_zone_id → zone.zone_id	상품 보관 위치
orders.pickup_slot_id → pickup_slot.slot_id	주문 픽업 슬롯
orders.assigned_unit_id → robot_unit.unit_id	주문 담당 로봇 세트
order_item.order_id → orders.order_id	주문에 포함된 상품 목록
order_item.product_id → product.product_id	주문 상품 정보
display_item.product_id → product.product_id	진열 대상 상품
display_item.assigned_unit_id → robot_unit.unit_id	진열 담당 로봇 세트
robot.unit_id → robot_unit.unit_id	PICKY와 COBOT 세트 연결
robot.current_task_id → task.task_id	로봇이 현재 수행 중인 작업
task.order_id → orders.order_id	task가 속한 주문

task.order_item_id → order_item.item_id	상품별 주문 task 연결
task.display_item_id → display_item.display_item_id	진열 task 연결
task.assigned_robot_id → robot.robot_id	task 담당 로봇
task.source_zone_id → zone.zone_id	작업 출발/기준 구역
task.target_zone_id → zone.zone_id	작업 목표 구역
task_event.task_id → task.task_id	작업 이벤트 대상 task
task_event.robot_id → robot.robot_id	작업 이벤트 발생 로봇
exception_log.robot_id → robot.robot_id	예외 발생 로봇
exception_log.task_id → task.task_id	예외 관련 task
exception_log.order_id → orders.order_id	예외 관련 주문

7. 주문 task 생성 기준

7-1. 일반 주문 task 흐름

순서	task_type	담당 로봇	기준
1	MOVE_TO_PROD UCT	PICKY	주문 상품 보관 구 역으로 이동
2	SORTING_AND_ LOAD	COBOT	주문 상품을 PICKY에 적재
3	MOVE_TO_PROD UCT	PICKY	다음 주문 상품 보 관 구역으로 이동

4	SORTING_AND_LOAD	COBOT	다음 주문 상품을 선별해 PICKY에 적재
5	MOVE_TO_PICKUP	PICKY	모든 상품 적재 후 픽업 구역으로 이동
6	INSPECTION	COBOT	주문 전체 상품 검수
7	UNLOAD	COBOT	픽업 슬롯에 상품 하차

상품이 여러 개면 MOVE_TO_PRODUCT 와 SORTING_AND_LOAD 는 order_item 기준으로 반복 생성된다.

반복 순서는 Fleet Manager가 상품 위치와 PICKY 현재 위치를 기준으로 계산한다.

7-2. 주문 task 예시

주문 ORD-0007 에 콜라, 과자, 물이 포함되어 있고 Fleet Manager가 과자 -> 콜라 -> 물 순서가 최적이라고 판단한 경우:

sequence_no	task_type	order_item	assigned_robot
1	MOVE_TO_PRODUCT	과자	PICKY1
2	SORTING_AND_LOAD	과자	COBOT1
3	MOVE_TO_PRODUCT	콜라	PICKY1
4	SORTING_AND_LOAD	콜라	COBOT1
5	MOVE_TO_PRODUCT	물	PICKY1
6	SORTING_AND_LOAD	물	COBOT1

7	MOVE_TO_PICK UP	없음	PICKY1
8	INSPECTION	주문 전체	COBOT1
9	UNLOAD	주문 전체	COBOT1

8. 진열 task 생성 기준

8-1. 진열 task 흐름

순서	task_type	담당 로봇	기준
1	MOVE_TO_STOCK	PICKY	창고 구역으로 이동
2	SORTING_AND_LOAD	COBOT	창고 구역 상품 중 진열할 상품을 PICKY에 적재
3	MOVE_TO_DISPLAY	PICKY	진열할 상품의 진열 구역으로 이동
4	DISPLAY_SCAN	COBOT	진열대 빈자리 탐색
5	DISPLAY_PLACE	COBOT	대상 상품을 진열대에 진열

DISPLAY_PLACE 가 SUCCESS 가 되면 `product.stock_qty` 를 증가시킨다.

진열 요청이 여러 상품으로 나뉘는 경우 `display_item` 을 상품별로 생성하고, Fleet Manager가 각 `display_item` 기준으로 task 흐름을 만든다.

8-2. 진열 수량 정책

조건	처리
LLM 명령에 수량이 있음	<code>requested_quantity</code> 에 저장
LLM 명령에 수량이 없음	<code>requested_quantity=NULL</code> , <code>display_policy=ALL_PROCESSED</code>

requested_quantity가 있음	지정된 수량만큼 진열
requested_quantity가 없음	COBOT이 실제 처리 완료한 대상 상품 전체를 진열
실제 처리 완료 수량	processed_quantity에 저장
최종 재고 증가 수량	stock_delta에 저장
재고 증가 기준	DISPLAY_PLACE SUCCESS 시 product.stock_qty += stock_delta

processed_quantity는 Vision이 한 번에 감지한 수량이 아니라, COBOT이 실제 집기/놓기/검수 완료한 수량이다.

8-3. 진열 데이터 예시

수량이 있는 명령:

```

1 {
2   "product_id": 1,
3   "requested_quantity": 5,
4   "processed_quantity": null,
5   "stock_delta": 5,
6   "display_policy": "REQUESTED_QUANTITY",
7   "status": "REQUESTED",
8   "assigned_unit_id": 1
9 }
```

수량이 없는 명령:

```

1 {
2   "product_id": 1,
3   "requested_quantity": null,
4   "processed_quantity": null,
5   "stock_delta": null,
6   "display_policy": "ALL_PROCESSED",
7   "status": "REQUESTED",
8   "assigned_unit_id": 1
9 }
```

SORTING_AND_LOAD / DISPLAY_PLACE 진행 중 실제 7개를 처리한 경우:

```

1 {
2   "product_id": 1,
3   "requested_quantity": null,
4   "processed_quantity": 7,
5   "stock_delta": 7,
6   "display_policy": "ALL_PROCESSED",
7   "status": "IN_PROGRESS",
8   "assigned_unit_id": 1
9 }
```

DISPLAY_PLACE 완료 후 진열이 끝난 경우:

```
1 {  
2   "product_id": 1,  
3   "requested_quantity": null,  
4   "processed_quantity": 7,  
5   "stock_delta": 7,  
6   "display_policy": "ALL_PROCESSED",  
7   "status": "COMPLETED",  
8   "assigned_unit_id": 1  
9 }
```

9. 상태 관리 기준

9-1. robot_status 기준

robot_status	의미
OFFLINE	통신 단절
IDLE	신규 작업 가능
BUSY	작업 수행 중, 신규 작업 불가
CHARGING	충전 중
EMERGENCY_STOP	긴급 정지 상태
ERROR	로봇 오류 상태

`robot_status` 는 Fleet Manager가 task 배정 가능 여부를 판단할 때 사용하는 큰 상태이다.

9-2. picky_state 기준

picky_state	의미
CHARGING	충전 중
STANDBY	대기/작업 가능 상태
MOVING_TO_PRODUCT	주문 상품 구역으로 이동 중
WAITING_FOR_COBOT	COBOT 작업 수행 대기 중
MOVING_TO_PICKUP	픽업 구역으로 이동 중
MOVING_TO_STOCK	창고 구역으로 이동 중
MOVING_TO_DISPLAY	진열 구역으로 이동 중
RETURNING	대기/충전 구역 복귀 중

DOCKING	충전 도크 정밀 진입 중
ERROR_RECOVERY	주행 오류 복구 중

9-3. cobot_state 기준

cobot_state	의미
STANDBY	작업 시작 전 기본 상태
SORTING	상품을 찾고 선별 중
LOADING	선별한 상품을 PICKY에 상차 중
INSPECTING	상품 검수 중
UNLOADING	픽업 슬롯 하차 중
SCANNING	진열대 빈자리 탐색 중
PLACING	진열 상품 진열 중
STOWING_ARM	로봇팔 기본 자세 복귀 중
SAFETY_STOPPED	안전 정지

10. 주의 사항

- Fleet Manager는 task와 robot 상태를 저장하고 Web Gateway에 제공한다.
- Fleet Manager는 DB 읽기/쓰기, task 생성, task 배정, TrafficManager 판단, State Machine 명령 전달을 담당한다.
- PICKY와 COBOT은 Web Gateway와 직접 통신하지 않는다.
- 주문 상품 목록은 `order_item`에 저장한다.
- 진열 상품 목록은 `display_item`에 저장한다.
- task는 주문 작업이면 `order_item_id`를 참조하고, 진열 작업이면 `display_item_id`를 참조한다.
- 진열 핵심 데이터는 `display_item`에 저장하고, task는 `display_item_id`로 연결한다.
- 긴급 정지는 `robot_status=EMERGENCY_STOP`으로 표현한다.
- `detected_quantity`는 사용하지 않는다. 최신 컬럼명은 `processed_quantity`이다.
- `ALL_DETECTED`는 사용하지 않는다. 최신 정책명은 `ALL_PROCESSED`이다.